

SMART VILLAGE

An OpenGL-Based Computer Graphics Project

Project Report

Course: CSE 358 (Computer Graphics & Multimedia Lab)

Submitted To:

Audity Ghosh & Any Chowdhury

Submitted By (Group Members):

Tamim Tanjil Molla	ID: 0432310005101100
Sadman Shahriar Sadaf	ID: 0432320005101086
Mehedi Hasan	ID: 0432320005101082
Syed Shihab Uddin	ID: 0432320005101062

Date of Submission: 19 May, 2026.

1. Abstract

Smart Village is an interactive computer graphics application developed using C++ and OpenGL (with GLUT/free GLUT). The project simulates a visually rich and animated rural village scene that incorporates natural elements, eco-friendly infrastructure, and dynamic environmental transitions. The application presents a modular codebase in which each scene component — mountains, houses, trees, wind turbines, clouds, sun, moon, and stars — is encapsulated within its own dedicated function, promoting readability, reusability, and maintainability.

The most distinctive feature of the project is its real-time day-night mode switching, controlled by a Boolean flag (`isDay`), which transforms the scene from a bright, sun-lit daytime landscape to a serene, star-filled night. Animation is driven by a timer-based callback that smoothly rotates wind turbine blades and drifts clouds horizontally across the sky. User interaction is handled through both keyboard and mouse input, making the project a demonstration of interactive computer graphics principles.

2. Introduction

Computer Graphics is a core discipline in modern computer science that bridges mathematical modeling and visual representation. It enables developers to simulate real-world environments and build interactive digital experiences. Among its many application areas, scene simulation — particularly animated, interactive environments — serves as an ideal vehicle for demonstrating core OpenGL concepts.

This project, Smart Village, was conceived and developed as part of a Computer Graphics course assignment. The objective was to design and implement an animated 2D scene using OpenGL primitives, transformations, and event-driven programming. The theme — a smart, eco-friendly village — was chosen to reflect both environmental awareness and the practical application of renewable energy concepts within a graphical context.

The project challenges the developer to combine multiple OpenGL techniques: primitive drawing with `GL_TRIANGLES`, `GL_QUADS`, `GL_POLYGON`, and `GL_TRIANGLE_FAN`; transformation using `glTranslatef`, `glRotatef`, and `glScalef`; color management with `glColor3f`; animation with `glutTimerFunc`; and event handling via keyboard/mouse callbacks.

3. Project Overview

Project Title	Smart Village
Language	C++
Graphics API	OpenGL (GLUT / freeGLUT)
Project Type	2D Animated Interactive Scene
Course	Computer Graphics
Key Themes	Eco-friendly village, Day/Night cycle, Wind energy, Animation
Interaction	Keyboard and Mouse event-driven controls
Animation	Timer-based cloud movement and windmill blade rotation

The application renders a panoramic village landscape that fills the entire window. The scene is divided into distinct visual layers — sky, mountains, ground, buildings, and vegetation — to create a sense of depth. Two wind turbines stand tall in the scene, their blades slowly rotating, symbolising renewable energy. Houses with blue walls, red roofs, and window details line up the ground. Trees and rolling hills add natural variety. Clouds drift lazily across the sky, and the sun or moon occupies the upper portion of the scene depending on the current time mode.

4. Objectives

The following objectives guided the design and development of Smart Village:

1. Apply OpenGL primitive drawing functions to construct a complete, visually appealing scene.
 2. Implement modular code design by separating each scene element into its own function.
 3. Demonstrate real-time animation through timer callbacks controlling cloud drift and windmill rotation.
 4. Implement a day-night switching mechanism that dynamically changes the scene background, celestial bodies, and tree/ground colors.
 5. Integrate user interaction through keyboard and mouse event handlers.
 6. Explore the concept of a smart eco-friendly village through graphical representation of wind turbines and organized housing.
 7. Fix and improve the code progressively — including resolving duplicate function naming (Sun ()), adding Moon (), and implementing stars for the night scene.
-

5. Scene Elements and Visual Design

The Smart Village scene is composed of multiple carefully designed graphical elements. Each element is implemented as a standalone function, called from within the display () callback.

5.1 Sky and Background

During the day, the sky is rendered as a solid bright purple-blue background (RGB approximately 0.4, 0.2, 1.0), giving the scene a vivid, stylized appearance. At night, the sky transitions to a deep navy (approximately 0.05, 0.05, 0.2), conveying a calm nocturnal atmosphere. This background is drawn first as a full-screen quad to serve as the canvas for all other elements.

5.2 Mountains

Two pairs of mountains frame the left and right edges of the scene. Each mountain is constructed using GL_TRIANGLES. The lower portions are rendered in a dark grey-brown (simulating rock), while snow-capped peaks are drawn in white at the apex, creating visual contrast and a sense of elevation.

5.3 Houses

The village contains multiple houses positioned along the ground line. Each house consists of a rectangular body (light blue walls) drawn with GL_QUADS, a triangular red roof drawn with GL_TRIANGLES, and window/door details using smaller quads in grey and yellow tones. During the night mode, windows emit a warm yellow glow, simulating interior lighting — a subtle detail that enhances the night atmosphere.

5.4 Wind Turbines

Three wind turbines are distributed across the scene — a large central turbine flanked by two smaller ones. Each turbine consists of a cylindrical pole (a thin quad) and three rotating blades implemented as elongated triangles. The blades are animated using a rotation angle variable updated by the timer callback, and OpenGL matrix transformations (glPushMatrix, glPopMatrix, glRotatef) are used to rotate them about the turbine hub.

5.5 Trees

The scene includes two types of trees: rounded deciduous trees (bright green during the day, darker green at night) built from GL_TRIANGLE_FAN circles and rectangular trunks, and stylised conifer trees formed from stacked triangles. These are scattered around the houses and hills to add natural variety.

5.6 Clouds

Clouds are constructed by overlapping multiple ellipses (circles of varying sizes drawn with GL_TRIANGLE_FAN). They move horizontally across the screen — animated via a cloud offset variable updated by the timer. When a cloud drifts off the right edge of the screen, it wraps around to reappear on the left, creating a seamless looping animation.

5.7 Sun and Moon

During the day, a bright yellow circle (drawn with `GL_TRIANGLE_FAN`) represents the sun in the upper-centre of the sky. During the night, the sun is replaced by a crescent moon, implemented by overlaying a slightly offset circle of the night-sky color atop a white circle. Stars are also rendered at fixed positions during the night as small white point-like dots using `GL_POINTS` or small quads, populating the dark sky.

5.8 Ground

The ground is a flat green rectangle spanning the bottom of the scene. A slightly darker green or black strip forms the road/path in front of the houses. During night mode, the ground darkens subtly to complement the overall palette shift.

6. Project Output Screenshots

The following screenshots were captured from the running application, demonstrating both the daytime and nighttime rendering modes:

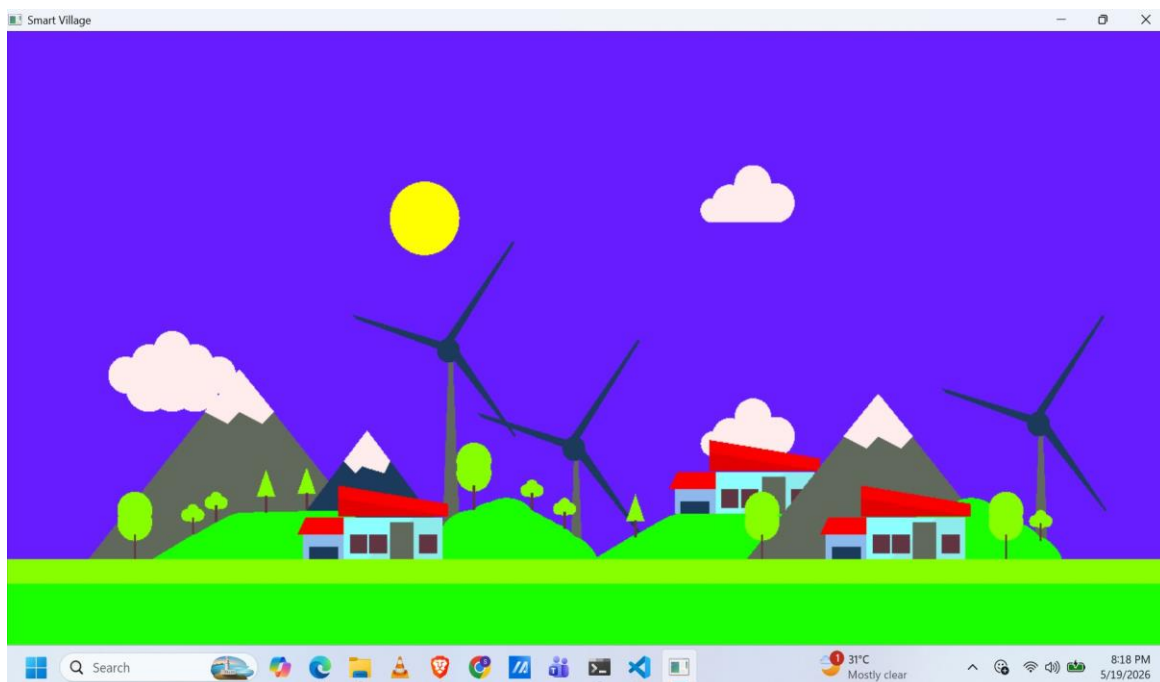


Figure 1: Smart Village — Daytime Scene (Purple sky, sun, bright green foliage, illuminated wind turbines)

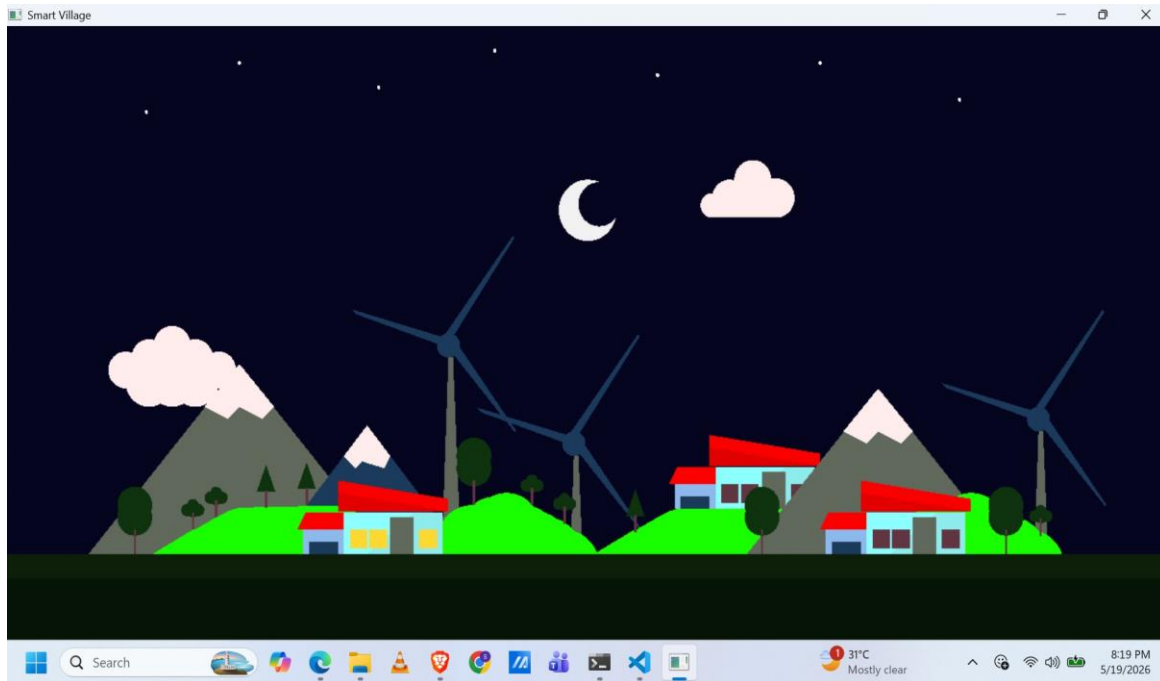


Figure 2: Smart Village — Nighttime Scene (Deep navy sky, crescent moon, stars, dark foliage, glowing house windows)

As shown in Figure 1, the daytime scene features a vivid purple sky, a bright yellow sun, white clouds drifting across the sky, lush green trees, and three animated wind turbines. The houses display blue facades with red roofs and visible window details.

Figure 2 demonstrates the seamless night transformation — the sky darkens, stars appear, the sun is replaced by a crescent moon, the trees take on a darker forest-green hue, and house windows emit a warm yellow glow to simulate interior lighting. The wind turbines continue their rotation in the night scene, maintaining the animation continuity.

7. Key Features

#	Feature	Description
1	Day/Night Toggle	Real-time scene transformation via isDay boolean flag. Sky color, celestial bodies, ground, and tree colors all change.
2	Windmill Animation	Three wind turbines with continuously rotating blades using glRotatef() and timer callbacks.
3	Cloud Movement	Clouds drift horizontally across the screen with seamless wrap-around animation.
4	Modular Code Design	Each scene element (Sun, Moon, Cloud, House, Tree, Mountain, etc.) has its own dedicated function.
5	Night Stars	Multiple stars rendered across the night sky using small primitives at fixed positions.
6	Crescent Moon	Night scene features a correctly shaped crescent moon using overlapping circle technique.
7	Window Glow	House windows display yellow color at night, simulating interior lighting.
8	Keyboard Interaction	Keyboard callbacks allow the user to switch modes or control scene elements.
9	Mouse Interaction	Mouse event handler responds to click events within the scene.
10	Layered Scene Depth	Mountains, hills, houses, and foreground elements are drawn in order to simulate depth.

8. Technical Implementation

8.1 Project Structure

The project follows a single-file C++ structure typical of introductory OpenGL applications. The key functions and their roles are:

- `main()` — Initializes GLUT, creates the window, registers callbacks, and enters the main event loop.
- `display()` — The primary rendering callback. Clears the screen and calls all scene element functions in the correct layering order.
- `init()` — Sets up the OpenGL viewport, projection matrix (orthographic), and background color.
- `move_right()` — Timer callback function that increments the cloud offset and windmill blade rotation angle, then triggers a redisplay.
- `keyboard()` — Handles keyboard events (e.g., pressing 'D'/'d' for day, 'N'/'n' for night, 'Q'/'q' or ESC to quit).
- `mouse()` — Handles mouse click events for additional user interaction.
- Individual element functions — `Sun()`, `Moon()`, `Cloud()`, `Mountain()`, `House()`, `Tree()`, `WindTurbine()`, `Ground()`, `Stars()` — each responsible for drawing one scene component.

8.2 Coordinate System

The scene uses an orthographic projection with a coordinate system typically set to (0, width) on the X-axis and (0, height) on the Y-axis, where the origin (0,0) is at the bottom-left corner. This makes positioning intuitive — higher Y values place objects higher on screen.

8.3 Animation Mechanism

Animation is implemented via `glutTimerFunc()`, which schedules the `move_right()` callback at regular intervals (e.g., every 16 milliseconds, approximating 60 FPS). Inside `move_right()`, the cloud offset variable is incremented and the windmill angle is updated. `glutPostRedisplay()` is called to request a screen refresh, and `glutTimerFunc()` is called again to schedule the next tick, creating a continuous animation loop.

8.4 Day-Night Switch

A global boolean variable `isDay` controls the scene mode. When `isDay` is true, the daytime rendering logic is used (bright sky, sun, bright trees). When false, the nighttime rendering is applied (dark sky, moon, stars, dark trees, glowing windows). This flag is toggled in response to keyboard input, and a screen redisplay is triggered immediately.

8.5 Windmill Blade Rotation

Each windmill hub is positioned using `glTranslatef()`. Before drawing the blades, `glPushMatrix()` saves the current transformation state. `glRotatef(angle, 0, 0, 1)` applies the rotation around the Z-axis (screen-normal axis for 2D). The three blades are drawn at 120-degree offsets from each other. After drawing, `glPopMatrix()` restores the state, ensuring subsequent drawing commands are unaffected.

9. Challenges and Solutions

Duplicate Sun() Function	Initially had two functions named Sun(), causing a compilation error. Fixed by renaming or removing the duplicate and consolidating sun-drawing logic into a single function.
Night Sky Elements	Adding Moon and Stars required implementing a crescent shape (two overlapping circles) and scattering small primitives across the sky during night mode.
Smooth Cloud Loop	When clouds drifted off-screen, they left an empty gap. Solved by resetting the cloud offset to a negative starting position when it exceeds the screen width, creating a seamless loop.
Windmill Centering	Applying rotation required careful use of glPushMatrix/glPopMatrix and translating to the hub center before rotating, then translating back to draw blades at the correct positions.
Color Palette Shifts	Ensuring all elements changed appropriately between day and night required checking the isDay flag inside each relevant drawing function and applying conditional color choices.

10. Learning Outcomes

Developing Smart Village provided hands-on experience with the following concepts and skills:

8. OpenGL primitive drawing using GL_TRIANGLES, GL_QUADS, GL_POLYGON, GL_TRIANGLE_FAN, and GL_POINTS.
9. Understanding and applying orthographic projection for 2D scene rendering.
10. Implementing real-time animation using GLUT timer callbacks (glutTimerFunc).
11. Using OpenGL matrix stack operations (glPushMatrix, glPopMatrix, glTranslatef, glRotatef) for hierarchical transformations.
12. Designing modular, maintainable code by encapsulating each scene element in a dedicated function.
13. Event-driven programming through GLUT keyboard and mouse callback registration.
14. Managing global state variables (isDay, cloudOffset, windAngle) for scene control.
15. Debugging graphical issues such as Z-fighting, incorrect transformation order, and off-screen element handling.

11. Conclusion

Smart Village successfully demonstrates the application of core OpenGL and computer graphics principles to create an engaging, animated, and interactive 2D scene. The project goes beyond simple static rendering by incorporating real-time animation (rotating wind turbines and drifting clouds), a dynamic day-night transition system, and user-driven interaction through keyboard and mouse events.

The modular code architecture ensures that each visual element is self-contained and easy to modify or extend. The project's eco-friendly theme — centering on wind energy and a green village environment — adds contextual meaning to the technical exercise, connecting graphical output to real-world concepts.

Future enhancements could include: transitioning to OpenGL 3.x/4.x with shader-based rendering (GLSL); adding a sunrise/sunset gradient animation; incorporating rain or snow particle effects; expanding to 3D geometry; and adding an on-screen HUD displaying the current mode and frame rate.

Overall, Smart Village stands as a complete and polished demonstration of 2D OpenGL graphics, meeting all the objectives of the Computer Graphics course project requirement.

12. References

- Google Scholar - [Google Scholar](#)
- GLUT (OpenGL Utility Toolkit) Documentation. Mark J. Kilgard, Silicon Graphics.
- freeGLUT Project. <http://freeglut.sourceforge.net/>
- LearnOpenGL. (2014–present). <https://learnopengl.com/> — Tutorials on OpenGL concepts and techniques.
- OpenGL.org. Reference Documentation. <https://www.opengl.org/sdk/docs/>
- Foley, J.D., van Dam, A., Feiner, S.K., & Hughes, J.F. (1990). Computer Graphics: Principles and Practice (2nd ed.). Addison-Wesley.